中国大学生计算机设计大赛

软件应用与开发类作品设计和开发文档

作品编号：2026035952

作品名称：赵都云旅·云端图鉴智行指南

版本编号：V1.0

填写日期：2026 年 4 月 28 日

《赵都云旅·云端图鉴智行指南》设计与开发文档

目录

第一章　需求分析 3

（一）　项目背景 3

（二）　建设目标 3

（三）　用户角色分析 4

（四）　功能需求 4

（五）　非功能需求 5

第二章　概要设计 6

（一）　总体架构 6

（二）　技术选型 6

（三）　目录结构 7

（四）　核心业务流程 7

（五）　视觉与交互设计 8

第三章　详细设计 9

（一）　前端页面设计 9

（二）　后端接口设计 9

（三）　数据结构设计 11

（四）　路线规划算法设计 11

（五）　AI 模型设计 12

（六）　语音导游设计 12

（七）　地图与图片资源设计 12

（八）　安全设计 13

第四章　测试报告 14

（一）　测试目标 14

（二）　测试环境 14

（三）　测试用例 14

（四）　实际命令结果 15

（五）　测试结论 15

第五章　安装及使用 16

（一）　本地开发环境 16

（二） 本地启动步骤 16
（三） 生产部署步骤 17
（四） 关键环境变量 17
（五） 使用说明 18

第六章 项目总结 19
（一） 项目成果 19
（二） 创新点 19
（三） 商业变现模式 19
（四） 不足与展望 20

参考文献 21

第一章　需求分析

（一）　项目背景

邯郸是国家历史文化名城，拥有赵文化、成语典故、太极文化、古城水系、山水休闲和地方饮食等多元文旅资源。传统旅游网站通常以景点列表、攻略文章或静态路线为主，游客在实际决策时仍需要反复比较开放时间、地理位置、交通方式、吃饭时间和同行人偏好。对于第一次到访的游客，最常见的问题不是“有没有景点”，而是“怎样把景点和美食排成一条顺路、不累、能执行的路线”。

本项目以“赵都云旅·云端图鉴智行指南”为题，面向邯郸城市文旅场景构建一个集城市首页、AI 导游、路线规划、图文图鉴、地图展示和后台管理于一体的网站系统。系统将生成式 AI 与规则规划能力结合：规则层保证路线顺路、预算可控、节奏合理；大模型层补充更自然的讲解、路线理由和游客友好的表达；地图层使用 MapLibre 展示点位和路线顺序，降低商业地图接口成本。

图 **1**　项目标识

（二）　建设目标

项目建设目标包括以下几个方面：

1. 建立一个面向游客的邯郸文旅门户首页，提供景点、美食、主题路线和 AI 导游入口。
2. 将路线规划入口改造成直接与 AI 对话体验的形式，减少一开始大量表单选择带来的枯燥感。
3. 支持 AI 路线规划，用户只需描述游玩时长、同行人和偏好，系统即可生成可执行的图文路线。
4. 支持语音导游，游客可以通过文字或语音询问景点故事、附近美食和路线建议。
5. 使用 MapLibre 开源地图展示路线点位，避免依赖高成本商业地图接口。
6. 提供后台管理能力，支持景点、美食、Prompt 和基础数据维护。
7. 提供 Docker Compose 部署方案和完整服务器使用手册，便于上线演示和后续运维。

（三） 用户角色分析

系统主要面向三类用户：

表 **1** 用户角色与需求

用户角色	典型场景	核心需求
普通游客	第一次来邯郸或周末短途出行	快速知道去哪、怎么走、吃什么，路线要顺路、轻松、表达易懂。
内容管理员	维护景点、美食、导游提示内容	能登录后台查看数据概览，维护景点和餐饮信息，发布 Prompt 配置。
部署维护人员	将项目部署到服务器和域名	需要清晰的环境变量、Docker、Nginx、HTTPS、备份与更新说明。

（四） 功能需求

表 **2** 功能需求列表

模块	功能项	说明
首页门户	城市文旅首页	以城市旅游局官网和 OpenAI 风格为参考，展示城市形象、核心入口、智能行程和热门图鉴。
AI 路线规划	对话式收集偏好	用户通过自然语言输入需求，系统追问时长、兴趣、同行人、交通方式等关键条件。
路线结果	图文行程展示	输出每日主题、时间段、景点、美食、交通建议、预算、替代方案与风险提示。
地图展示	MapLibre 路线图	在规划结果顶部展示地图，标注景点和餐饮点位，体现路线顺序。
语音导游	文字/语音问答	游客可询问景点故事、附近美食、轻松路线、拍照建议等，系统以导游口吻回复。
图鉴内容	景点与美食详情	提供景点详情页、美食详情页、主题页、经典路线页和 FAQ 页面。
后台管理	内容维护	使用 Basic Auth 登录，维护景点、餐饮和 Prompt，查看数据概览。
反馈分享	分享与评价	行程生成后保存 session 和 shareId，支持分享和用户反馈。

（五） 非功能需求

1. 易用性：游客入口必须足够简单，首页不强迫用户先填写复杂表单，而是先提供自然语言输入和 AI 导游入口。
2. 可执行性：路线规划不能只给泛泛建议，必须包含点位顺序、时间段、餐饮安排、预算和风险提示。
3. 可维护性：项目采用 monorepo 结构，前端、后端、共享类型、规划引擎、数据层分离，便于迭代。
4. 安全性：后台接口使用 Basic Auth，生产环境中的 API key、数据库密码和后台密码均通过环境变量配置，禁止写入仓库。
5. 成本控制：地图使用 MapLibre 和开源底图，AI 调用可通过开关启停；没有模型 key 时可回退到规则规划。
6. 可部署性：提供 Docker Compose、Nginx、HTTPS、域名解析、备份和更新流程，适合云服务器快速上线。

第二章　概要设计

（一）　总体架构

系统采用前后端分离的全栈架构，代码以 pnpm workspace 和 Turborepo 组织。前端使用 Next.js App Router 提供门户页面、AI 导游页面、路线结果页面和后台页面；后端使用 NestJS 提供 REST API、流式路线生成、语音导游接口和后台接口；数据层使用 Prisma 连接 PostgreSQL，并用 Redis 作为缓存扩展预留；AI 能力通过火山方舟文本模型和豆包端到端实时语音能力接入。

游客浏览器 / 移动端

⇓

Next.js Web：城市首页、AI 导游、路线规划、地图结果、后台界面

⇓

NestJS API：内容查询、路线生成、流式输出、语音问答、后台管理

⇓

共享领域层：Zod Schema、TypeScript 类型、路线规划规则、Prompt 模板

⇓

PostgreSQL / Redis / MapLibre / 火山方舟文本模型 / 豆包实时语音

图 **2**　系统总体架构

（二）　技术选型

表 **3**　主要技术栈

层级	技术	选型原因
前端	Next.js 16、React 19、Tailwind CSS、Framer Motion	适合构建 SEO 页面与交互页面，支持服务端渲染、动效和组件化开发。
地图	MapLibre GL	开源、成本低，可展示真实地图和点位顺序，适合替代高成本商业地图接口。
后端	NestJS、Zod	模块化清晰，便于拆分控制器与服务；Zod 用于请求和响应结构校验。

数据层	Prisma、PostgreSQL、Redis	Prisma 提供类型安全 ORM，PostgreSQL 存储结构化文旅数据，Redis 预留缓存能力。
AI 能力	火山方舟、Doubao-Seed-2.0-mini、豆包端到端实时语音	文本模型增强路线表达和导游问答，实时语音提升沉浸式导览体验。
工程化	pnpm workspace、Turborepo、Vitest、Playwright、Docker Compose	支持多包协作、快速构建、自动化测试和容器化部署。

（三） 目录结构

项目目录结构如下：

Listing 1: 项目核心目录结构

```
apps/
  web       # Next.js frontend
  api       # NestJS backend
packages/
  shared    # Zod schemas and shared types
  planner   # rule-based route planner
  data      # Prisma schema and seed data
  prompts   # LLM prompts
  ui        # shared UI components
docs/
  api-integration.md
  architecture.md
  configuration.md
  deployment.md
```

（四） 核心业务流程

1. 路线规划流程

路线规划的核心流程为：

1. 用户在首页或 AI 导游页输入自然语言需求，例如“两天一夜，带父母，想看历史也想吃特色”。
2. 前端将自然语言与补充条件整理为统一的 PlanIntent 结构。
3. 后端使用 Zod 校验请求体，读取景点、美食、菜品和主题数据。

4. 规则规划引擎根据兴趣、预算、同行人、节奏和距离进行候选点评分，生成基础路线。
5. 如果启用 LLM，则将候选数据和基础路线发送给模型，模型只负责优先级建议、路线理由、摘要和提示文案，不凭空编造点位。
6. 系统保存 PlanSession，生成 shareId，并返回包含地图点位、日程、预算、替代方案和风险提示的结果。
7. 前端使用流式接口逐步显示规划进度，最终在结果页呈现完整图文路线。

2. 语音导游流程

语音导游包含两种能力：一是用户提问后的文本问答，二是实时语音或语音合成。浏览器端负责麦克风权限、录音和播放，后端通过 `/voice/guide` 调用文本模型生成导游式回答，通过 `/voice/tts` 或 `/voice/realtime` 接入豆包端到端实时语音模型。未配置语音密钥时，系统仍能以文字问答和浏览器朗读方式运行，保证演示稳定性。

（五） 视觉与交互设计

首页采用城市官网式信息结构与 OpenAI 风格的大留白、大标题、柔和动效结合的设计方式。首屏突出城市气质和“智能安排行程”入口，右上角提供 AI 导游按钮，进入后可选择“语音导游”和“AI 路线规划”。视觉上使用用户提供的邯郸实拍图作为头图，搭配项目标识、景点图鉴和美食图片，避免使用无关图片。

图 **3** 首页使用的邯郸实拍头图

第三章　详细设计

（一）　前端页面设计

前端位于 `apps/web`，采用 Next.js App Router。页面结构如下：

表 **4**　前端页面与功能

页面路径	页面名称	主要功能
`/`	首页	展示城市门户、智能行程入口、热门景点、美食图鉴、主题路线和 AI 导游入口。
`/guide`	AI 导游	支持语音导游和 AI 路线规划两种模式切换。
`/plan`	路线结果页	展示地图、每日行程、预算、替代方案、风险提示和继续微调入口。
`/poi/[slug]`	景点详情	展示景点图片、简介、标签、建议游玩时长和门票参考。
`/food/[slug]`	美食详情	展示门店或名吃详情、适合时间和推荐理由。
`/theme/[slug]`	主题玩法	按成语文化、赵文化、太极文化、夜游美食等主题聚合内容。
`/routes/[slug]`	经典路线	展示固定路线模板与路线亮点。
`/admin`	后台管理	管理景点、美食和 Prompt 内容。

关键组件包括 PlanForm、PlanResultView、RouteMap、VoiceGuide、GuideModeSwitcher、SiteHeader 和 SiteFooter。其中 RouteMap 负责 MapLibre 地图渲染，VoiceGuide 负责语音问答交互，PlanForm 负责对话式收集路线偏好。

（二）　后端接口设计

后端位于 `apps/api`，所有接口统一使用 `/api/v1` 前缀。线上同站点代理地址为 `https://qiuyu.online/api/proxy`。公开接口无需鉴权，后台接口使用 Basic Auth。

表 **5**　主要 **API** 接口

接口	类型	功能说明
GET/health	公开	健康检查，返回数据库和服务状态。
GET/pois	公开	查询景点列表，支持 district、theme、q 筛选。
GET/pois/:slug	公开	查询景点详情。
GET/foods	公开	查询美食目录，包含门店和菜品。
GET/foods/:slug	公开	查询美食详情。
GET/themes	公开	查询主题列表。
GET/themes/:slug	公开	查询主题详情及关联景点、美食。
GET/search	公开	全局搜索景点、美食和主题。
POST/plans/generate	公开	生成完整路线规划结果。
POST/plans/generate-stream	公开	以 Server-Sent Events 方式流式输出路线规划进度和最终结果。
POST/plans/refine	公开	基于上一版 session 微调路线。
GET/plans/share/:shareId	公开	读取分享路线。
POST/voice/guide	公开	文本或语音导游问答。
POST/voice/tts	公开	语音合成，优先使用豆包实时语音，失败时返回兜底信息。
POST/feedback	公开	提交路线反馈。
GET/admin/overview	后台	查看景点、餐饮、Prompt 数量概览。
GET/admin/prompts	后台	查看 Prompt 配置。
POST/admin/pois	后台	新增景点。
PUT/admin/pois/:id	后台	更新景点。
POST/admin/foods	后台	新增餐饮。
PUT/admin/foods/:id	后台	更新餐饮。

接口	类型	功能说明
`POST/admin/prompts/publish`	后台	发布 Prompt 配置。

（三） 数据结构设计

数据库使用 PostgreSQL，Prisma Schema 位于 `packages/data/prisma/schema.prisma`。核心实体包括 SourceRecord、Poi、FoodVenue、Dish、Theme、PlanSession、UserFeedback 和 PromptConfig。

表 **6** 核心数据表

数据表	主要字段与用途
SourceRecord	记录数据来源、发布者、链接、抓取时间，为景点和美食内容提供可追溯依据。
Poi	存储景点名称、slug、区县、经纬度、标签、主题、简介、游玩时长、门票、交通提示和审核状态。
FoodVenue	存储餐饮门店名称、经纬度、人均、辣度、招牌菜、适合人群和文化故事。
Dish	存储地方菜品、口味、文化标签、关联门店和适合用餐时间。
Theme	存储主题玩法，如成语文化、赵文化、太极文化、夜游美食等。
PlanSession	存储用户意图、生成结果、模型元数据和分享 ID。
UserFeedback	存储用户评分、是否有用和评价内容。
PromptConfig	存储 Prompt 名称、版本、系统提示词和发布状态。

（四） 路线规划算法设计

路线规划采用“规则规划 + 模型增强”的混合策略。规则层首先根据用户意图对景点和餐饮点位评分，再按每日节奏和交通方式拼装日程。景点评分综合考虑兴趣匹配、同行人适配、必去点、城市区域、低步行约束和 AI 优先级建议，可抽象为：

$$S(p) = S_{interest} + S_{crowd} + S_{must} + S_{city} - P_{walking} + B_{AI} \tag{1}$$

其中，$S_{interest}$ 表示景点标签与用户兴趣的匹配度，S_{crowd} 表示景点是否适合家庭、父母、情侣等同行人，S_{must} 表示用户指定必去点加权，$P_{walking}$ 表示少走路场景下对高强度景点的惩罚，B_{AI} 表示模型建议优先级带来的增益。

交通时间采用经纬度计算距离，再根据交通方式估算换乘时间。对于步行为主、公交、打车和自驾四种方式，系统分别使用不同的时间系数，避免把远距离点位排得过紧。餐饮选择则根据预算、人均、同区县和附近美食 ID 综合排序。

（五） AI 模型设计

系统文本模型默认配置为 `doubao-seed-2-0-mini-260215`，通过火山方舟 OpenAI 兼容接口调用。路线规划中的大模型并不直接生成完整路线，而是在规则路线基础上输出以下增强内容：

1. 景点和餐饮点位优先级；
2. 更自然的 tripTitle、summary 和 whyThisPlan；
3. 每日主题名称；
4. 雨天、少走路、预算等替代方案；
5. 每个景点或餐饮段落的游客化理由；
6. 必要的行程风险提示。

为避免模型编造点位，Prompt 明确要求只能从候选 POI 和餐饮实体中选择，并使用 JSON 输出。后端再用 Zod 对模型结果二次校验，过滤不在候选集内的 ID。若模型调用失败或未配置 API key，系统自动回退到规则规划结果。

（六） 语音导游设计

语音导游模块包含 `/voice/guide`、`/voice/tts` 和 `/voice/realtime` 三部分。文本问答接口通过系统提示词约束模型角色：它是“赵都云旅·云端图鉴智行指南”的邯郸语音导游，回复应亲切、简短、具体，避免开发者术语。实时语音网关使用 WebSocket 将浏览器音频转发给豆包端到端实时语音服务，并把 ASR、Chat、TTS 事件返回给前端。

该模块特别强调游客体验：界面不展示“浏览器听写”“服务连接失败”等开发者提示，而用自然语言呈现可继续输入、继续提问的状态，避免影响游客信任感。

（七） 地图与图片资源设计

地图模块使用 MapLibre GL，底图可配置为 OpenFreeMap 样式地址。结果页将地图放在规划内容上方，点位顺序和文本行程保持一致。图片资源统一存放

在前端 `public` 目录，包括项目标识、用户实拍头图、景点图片和美食图片。为保证图文并茂，项目已整理真实相关图片，避免使用与景点无关的占位图。

(a) 丛台公园

(b) 广府古城

(c) 邯郸美食

图 **4**　项目中的景点与美食图文资源

（八）　安全设计

系统安全设计主要包括：

1. 环境变量隔离：数据库地址、后台密码、AI Key、语音 Key 等仅写入 `.env`，仓库只保留 `.env.example`。
2. 后台鉴权：后台接口统一使用 Basic Auth，由 `AdminGuard` 校验用户名和密码。
3. 参数校验：所有关键请求使用 Zod Schema 校验，避免非法结构进入业务逻辑。
4. 错误脱敏：AI provider 报错时会对 Bearer Token 进行脱敏截断，避免密钥泄露。
5. 网络隔离：生产部署建议 API、PostgreSQL 和 Redis 只监听 `127.0.0.1` 或 Docker 内网，再由 Nginx 代理 Web。

第四章　测试报告

（一）　测试目标

本项目测试目标包括：共享 Schema 能正确约束输入；路线规划引擎能生成结构化可执行路线；Web 前端能完成生产构建；API 后端能完成 TypeScript 构建；核心页面路由能被 Next.js 正确识别和输出。

（二）　测试环境

表 **7**　测试环境

项目	环境
操作系统	macOS，本地开发目录为 handan-llm-travel 项目根目录。
Node 包管理器	pnpm 10.28.2
前端框架	Next.js 16.2.3、React 19.2.4
后端框架	NestJS 11
测试工具	Vitest 3.2.4、Playwright 1.55.0
编译工具	TypeScript、Next.js production build、Nest CLI

（三）　测试用例

表 **8**　关键测试用例

测试对象	测试内容	结果
`@handan/shared`	Schema 单元测试，验证共享类型和请求结构约束	1 个测试通过。
`@handan/planner`	输入两天一夜历史美食游意图，验证返回 tripTitle、days 和 segments	1 个测试通过。
Web 前端	执行 `pnpm--filterwebbuild`，验证生产构建和静态页面生成	构建成功，生成首页、导游页、路线页、FAQ、后台页等页面。

API 后端	执行 `pnpm--filterapibuild`，验证 NestJS 后端 TypeScript 编译	构建成功。

（四） 实际命令结果

本次文档生成前执行了以下验证命令：

Listing 2: 测试与构建命令

```
pnpm --filter @handan/shared test
pnpm --filter @handan/planner test
pnpm --filter web build
pnpm --filter api build
```

结果如下：

1. `@handan/shared`：Vitest 显示 1 个测试文件、1 个测试用例全部通过。
2. `@handan/planner`：Vitest 显示 1 个测试文件、1 个测试用例全部通过。
3. Web 构建：Next.js 16.2.3 编译成功，完成 TypeScript 检查，生成 10 个静态页面，并识别动态路由。
4. API 构建：NestJS 后端构建命令正常退出，无 TypeScript 编译错误。

（五） 测试结论

从测试结果看，当前项目的共享类型、路线规划核心逻辑、前端生产构建和后端生产构建均可正常运行，满足课程设计和比赛答辩演示的基本稳定性要求。后续如果进入正式运营阶段，建议继续补充 E2E 测试、接口压力测试、实时语音端到端测试和数据库迁移回归测试。

第五章　安装及使用

（一）　本地开发环境

本地开发建议准备以下环境：

表 **9**　开发环境清单

工具	说明
Node.js	建议 20 LTS 或更高版本。
pnpm	项目使用 pnpm 10.28.2 管理 monorepo 依赖。
Docker Desktop	本地启动 PostgreSQL 和 Redis。
PostgreSQL	生产环境数据库，开发环境可通过 Docker Compose 启动。
Redis	预留缓存和会话扩展能力。
Git	代码版本管理和 GitHub 推送。
VS Code / WebStorm	前端和后端开发编辑器。

（二）　本地启动步骤

1. 安装依赖：

```
pnpm install
```

2. 准备环境变量：

```
cp .env.example .env
```

3. 启动数据库和缓存：

```
docker compose up -d db redis
```

4. 初始化数据库：

```
pnpm db:generate
pnpm db:migrate
pnpm db:seed
```

5. 启动开发服务：

```
pnpm dev
```

默认访问地址为：Web 前端 `http://localhost:3000`，API 后端 `http://localhost:3001/api/v1`。

（三） 生产部署步骤

生产环境推荐使用 Docker Compose 和 Nginx：

1. 拉取项目：

```
git clone https://github.com/hanabinano/handan-llm-travel.git
cd handan-llm-travel
cp .env.example .env
```

2. 修改 `.env`，重点配置站点地址、数据库、Redis、后台密码、AI Key 和语音 Key。
3. 构建并启动容器：

```
docker compose build
docker compose up -d db redis
docker compose up -d api web
```

4. 初始化数据库：

```
docker compose exec api pnpm --filter @handan/data db:migrate
docker compose exec api pnpm --filter @handan/data db:seed
```

5. 使用 Nginx 将域名代理到 Web 容器监听端口，并通过 Certbot 配置 HTTPS。
6. 检查 `/api/proxy/health`、首页、AI 导游页、路线生成页和后台登录是否正常。

（四） 关键环境变量

表 **10** 关键环境变量

变量	说明
`NEXT_PUBLIC_SITE_URL`	网站公网地址，用于 SEO 和分享链接。
`NEXT_PUBLIC_API_BASE_URL`	前端访问后端 API 的基础地址，Docker 内部通常为 `http://api:3001/api/v1`。
`NEXT_PUBLIC_MAP_PROVIDER`	地图模式，推荐 `maplibre`。

NEXT_PUBLIC_MAPLIBRE_STYLE_URL	MapLibre 底图样式地址。
DATABASE_URL	PostgreSQL 连接地址。
REDIS_URL	Redis 连接地址。
ADMIN_USERNAME、ADMIN_PASSWORD	后台登录账号和密码。
PLANNER_USE_LLM	是否启用 LLM 路线增强。
ARK_API_KEY、ARK_MODEL	火山方舟文本模型配置。
VOICE_REALTIME_APP_ID、VOICE_REALTIME_API_KEY	豆包端到端实时语音配置。

（五） 使用说明

游客使用流程如下：

1. 打开首页，先查看城市推荐、景点图鉴和美食入口。
2. 在首页“智能安排行程”输入框中直接描述需求，如“周末带父母来邯郸，两天轻松一点，想看历史也想吃特色”。
3. 系统根据输入生成路线，过程中以流式方式显示规划进度。
4. 在路线结果页查看地图、每日安排、餐饮、预算和风险提示。
5. 若不满意，可继续输入“少走路”“多安排美食”“预算控制在 600 元”等指令微调。
6. 点击右上角 AI 导游，可在语音导游和 AI 路线规划之间切换，直接提问景点故事和路线建议。

管理员使用流程如下：

1. 访问 /admin。
2. 使用 .env 中配置的后台账号密码登录。
3. 查看数据概览，维护景点、美食、Prompt 配置。
4. 更新内容后检查首页、详情页和路线规划结果是否正常引用最新数据。

第六章　项目总结

（一）　项目成果

本项目完成了一个面向邯郸文旅场景的全栈网站系统，实现了从城市门户展示、AI 导游问答、路线规划、地图可视化到后台管理和服务器部署的完整闭环。相比传统静态旅游攻略，本项目的核心价值在于让用户用一句自然语言表达需求，系统自动结合景点、美食、交通、预算和游玩节奏生成一条可执行路线，并通过图文和地图降低理解成本。

项目已经具备以下成果：

1. 完成 Next.js 前端门户、AI 导游页、路线结果页、图鉴详情页和后台页面。
2. 完成 NestJS 后端接口、路线规划接口、流式输出接口、语音导游接口和后台管理接口。
3. 完成 Prisma 数据模型、种子数据、内容图鉴和用户反馈数据结构。
4. 完成 MapLibre 地图集成，降低地图服务成本。
5. 完成火山方舟文本模型和豆包实时语音能力接入设计，并提供兜底策略。
6. 完成 Docker Compose、Nginx、HTTPS、域名解析和更新备份部署文档。
7. 完成 GitHub 仓库 README、API 对接文档、配置说明和部署手册。

（二）　创新点

项目创新点主要体现在：

1. 将“城市旅游门户”与“AI 导游聊天”合并，首页既能展示城市形象，也能立即进入智能行程。
2. 使用规则规划和生成式 AI 混合方案，既保证路线可执行，又让文案更像真人导游。
3. 使用流式生成体验，避免用户等待路线生成时感到卡顿。
4. 使用 MapLibre 开源地图替代高成本商业地图接口，适合学生项目和中小文旅项目落地。
5. 引入语音导游模式，使系统不只是路线生成器，也能成为随问随答的城市讲解助手。

（三）　商业变现模式

如果项目继续商业化，可考虑以下方向：

1. 文旅局或景区定制服务：为城市、景区、文博场馆部署定制版 AI 导游和路线规划系统。
2. 商户推荐与转化：与本地餐饮、酒店、特产店合作，在不影响用户体验的前提下提供“顺路推荐”。
3. 旅游套餐分发：对接旅行社、研学机构、景区门票和民宿资源，按转化收取佣金。
4. SaaS 化输出：将系统抽象为可配置平台，支持不同城市导入景点、美食和语音讲解数据。
5. 数据服务：基于用户偏好、路线热度和反馈数据，为文旅运营提供热点分析和路线优化建议。

（四） 不足与展望

当前项目仍有进一步优化空间。第一，景点和美食数据仍以种子数据和人工维护为主，后续可增加更完整的数据采集、审核和更新流程。第二，路线规划目前以启发式规则为主，后续可加入真实道路距离、公共交通时间、营业时间校验和天气联动。第三，语音导游虽然已接入实时语音能力，但仍需要更多端到端场景测试，尤其是弱网、移动端权限和多轮对话稳定性。第四，商业化场景下还需要加强内容审核、日志监控、风控和数据隐私合规。

总体来看，“赵都云旅·云端图鉴智行指南”已经形成了一个可演示、可部署、可继续扩展的 AI 文旅助手原型。它既能服务游客的即时出行决策，也能为城市文旅数字化提供一个低成本、可复制的实践方向。

参考文献

[1] Vercel. Next.js Documentation. https://nextjs.org/docs. 访问日期：2026-04-29.

[2] NestJS. NestJS Documentation. https://docs.nestjs.com/. 访问日期：2026-04-29.

[3] Prisma. Prisma Documentation. https://www.prisma.io/docs. 访问日期：2026-04-29.

[4] MapLibre. MapLibre GL JS Documentation. https://maplibre.org/maplibre-gl-js/docs/. 访问日期：2026-04-29.

[5] Docker. Docker Compose Documentation. https://docs.docker.com/compose/. 访问日期：2026-04-29.

[6] 火山引擎. 火山方舟与豆包模型相关文档. https://www.volcengine.com/docs. 访问日期：2026-04-29.

[7] 项目团队. 赵都云旅·云端图鉴智行指南项目仓库与开发文档. https://github.com/hanabinano/handan-llm-travel. 2026.